

Cross-Architecture Layer Grafting: Asymmetric Multimodal and Multi-Domain Integration under Severe Hardware Constraints

Abstract

Training multimodal and multi-domain large language models (LLMs) traditionally demands massive computational clusters for joint pre-training or instruction alignment. Furthermore, retrofitting an existing language model with highly specialized capabilities often results in catastrophic forgetting or architectural rejection if the source and target networks are structurally disparate. In this paper, we introduce Cross-Architecture Layer Grafting (CALG), a systems-architecture methodology for the asymmetric, zero-shot structural merging of heterogeneous neural networks. As a feasibility proof, we synthesize a tripartite hybrid architecture designated as FusionLM-v0.1-alpha. We graft the vision encoder from Llama-3.2-11B-Vision-Instruct, projected task-vector deltas derived from Qwen2.5-Coder specialization, and projected task-vector deltas derived from Gemma-2-Math specialization directly into a Phi-4-mini-instruct (3.8B) text backbone. We demonstrate that this disjointed composite architecture compiles after runtime graph stabilization and executes stable inference under severe hardware constraints, peaking at 14.37 GB of VRAM and achieving a throughput of 24.0 tokens per second on a single consumer-grade Nvidia T4 GPU. While the raw structural graft exhibits an expected latent representation clash—necessitating a post-graft boundary healing phase—this work establishes the hardware, compilation, and structural feasibility of heterogeneous multi-domain model synthesis without full-scale backpropagation.

1 Introduction

The rapid evolution of foundational neural networks has increasingly favored monolithic architectures and compute-intensive alignment phases. When a base language model requires structural adaptation for new sensory modalities or domain-specific parameters, the prevailing paradigm dictates either visual instruction tuning across millions of multi-modal pairs or extensive multi-task continuous pre-training (CPT). These methods are economically restrictive, limiting true architectural experimentation to heavily funded institutions and restricting open-source researchers to fine-tuning existing homogeneous weights.

Concurrently, the open-source community has pioneered post-training model merging techniques—such as SLERP, Task Vectors, and TIES-Merging—to combine model weights without gradient updates. However, these operations are strictly mathematically bound to architectural isomorphism. They require identical layer counts, matching hidden dimensions, and shared vocabulary sizes. They are fundamentally incapable of merging a Qwen2.5-based parameter block

with a Phi-4 reasoning backbone, nor can they graft a dedicated Llama-3.2 vision network into a text model without re-initializing a massive projection matrix requiring cluster-scale compute.

To bridge this critical gap in compute-efficient systems architecture, we present Cross-Architecture Layer Grafting (CALG). This approach diverges from traditional model merging by treating disjoint open-source models not as immutable blocks of identical matrices, but as modular components that can be surgically bound together via localized linear transformations, singular value decomposition, and explicit affine projections.

Our work explicitly targets the extreme edge of hardware constraints, evaluating whether massive, multi-domain structural configurations can be instantiated and validated on standard, consumer-tier hardware. To validate CALG, we construct a highly specialized hybrid network, FusionLM-v0.1-alpha, sourced from four distinct open-source families:

1. **The Host Backbone:** The 3.8B parameter Phi-4-mini-instruct, serving as the central causal routing graph.
2. **The Vision Component:** The 11B parameter vision encoder decoupled from Llama-3.2-11B-Vision-Instruct, integrated via a static dimensional bridge.
3. **The Coding Component:** Specialized intermediate attention and MLP layer deltas extracted from Qwen2.5-Coder.
4. **The Mathematical Component:** Specialized intermediate attention and MLP layer deltas extracted from Gemma-2-Math.

The primary architectural and systems contributions of this paper are as follows:

- **Asymmetric Cross-Modal Projection:** We define a methodology for mapping the Llama-3.2 vision encoder into the Phi-4-mini text network using an isolated, zero-shot projection bridge, projecting a 7680-dimensional visual latent representation into the 3072-dimensional hidden space of the host model without triggering runtime exceptions.
- **Deep Layer Structural Injection:** We demonstrate the localized surgical swapping of intermediate transformer blocks (Layers 28-31). By utilizing delta extractions and dimensional projections, we physically integrate task-vector-derived parameter deltas derived from Gemma-2-Math and Qwen2.5-Coder specializations across fundamentally mismatched architectures, preserving the structural integrity of the host’s execution graph.
- **Hardware Constraint Validation:** We provide empirical proof that FusionLM-v0.1-alpha can sustain autoregressive graph compilation within a strict 15GB VRAM envelope, maintaining a throughput of over 24.0 tokens per second on an Nvidia T4 GPU.

We emphasize that this paper demonstrates *systems and compilation feasibility*, not zero-shot capability transfer. While the resulting raw compilation natively produces semantic incoherence due to expected manifold mismatch—a phenomenon we rigorously analyze through activation divergence and low-rank geometric controls—this study establishes the foundational physical scaffolding required to bypass cluster-tier pre-training and democratize heterogeneous architectural engineering.

2 Related Work

The conceptual foundation of Cross-Architecture Layer Grafting (CALG) intersects three heavily researched areas of contemporary deep learning: parameter-space weight merging, modular cross-modal alignment, and the study of latent space geometry within deep transformer networks.

2.1 Weight-Space Interpolation and Homogeneous Merging

Post-training weight space modification has emerged as a critical paradigm for combining specialized model capabilities without incurring the economic and environmental costs of full-scale multi-task training. Early approaches relied on linear or spherical interpolation (SLERP) to merge model parameters sharing a common ancestor.

This framework was formalized by Ilharco et al. [2] with the introduction of *Task Vectors*. A task vector isolates the direction of specialized knowledge acquisition by subtracting the parameters of a pre-trained base model from those of a fine-tuned variant:

$$\tau = W_{\text{tuned}} - W_{\text{base}}$$

Specialized domain capabilities can then be combined by executing weight-space addition ($W_{\text{new}} = W_{\text{base}} + \lambda\tau$). Advanced interpolation frameworks like TIES-Merging [5] and DARE [6] introduce multi-step pipelines to filter low-magnitude parameter shifts and enforce sparsity prior to summation.

The CALG Differentiation: Despite their empirical successes, these frameworks are strictly limited by the requirement of *architectural isomorphism*. They mathematically demand that the underlying parameter matrices share identical tensor shapes, layer counts, and vocabulary configurations. CALG bypasses this restriction entirely. Rather than executing element-wise arithmetic operations inside a single shared coordinate system, CALG operates asymmetrically at layer boundaries, employing Singular Value Decomposition (SVD) to compress and project task vectors across structurally incompatible computation graphs.

2.2 Modular Multimodal Alignment and Projection Frameworks

The integration of distinct sensory modalities within text-centric architectures was popularized by modular frameworks like LLaVA [3] and Flamingo [1]. These networks maintain a frozen visual feature extractor and map its output hidden states into a causal language model backbone via an intermediary projection module.

The CALG Differentiation: While LLaVA relies on a projection mechanism similar to the static linear bridge used in our framework, existing architectures require optimizing the projection parameters (θ_{proj}) over millions of image-text pairs during a substantial multi-modal pre-training phase. Without this initial optimization pass, standard frameworks fail to compile an operational forward pass. Furthermore, these architectures treat the internal text backbone as an unalterable black box.

Our work shifts this focus by evaluating the zero-shot physical constraints of combining a massive vision module with a compact text backbone under a strict 15GB VRAM limit. Crucially, CALG couples this cross-modal projection with simultaneous intermediate layer in-

jections from entirely separate text families, analyzing how multiple disjoint geometric spaces can be mechanically forced to interact within a single, highly constrained runtime environment.

2.3 Latent Space Geometry and Layer Independence

The viability of surgically modifying individual intermediate layers rests on recent structural insights into deep transformer networks. Investigations into layer redundancy, such as Short-GPT [4], demonstrate that intermediate blocks often exhibit high degrees of linear correlation and representation redundancy, proving that layers can be aggressively pruned without triggering catastrophic perplexity collapse.

CALG tests an extreme architectural hypothesis based on this foundational premise: if intermediate residual layers operate with high degrees of independence, can they be entirely replaced by dimensionally compressed, cross-family parameter blocks? By focusing our Qwen and Gemma delta injections exclusively on deep reasoning blocks ($L_{28} - L_{31}$), we exploit the network’s capacity to absorb structural variance without triggering runtime compilation faults in the early lexical parsing layers.

3 Methodology

The Cross-Architecture Layer Grafting (CALG) paradigm decomposes the challenge of multi-domain, heterogeneous model fusion into a three-stage topological pipeline: (1) Asymmetric Cross-Modal Translation, (2) Task Vector Extraction from donor networks, and (3) Rank-Truncated Dimensional Alignment for deep-layer skill injection.

3.1 Architectural Formulation and Active Parameter Allocation

We define the host causal reasoning backbone, M_{host} , as the Phi-4-mini-instruct (3.8B parameters) architecture. The host network processes information through a discrete set of transformer blocks $L = \{L_0, L_1, \dots, L_{31}\}$, operating within a strictly bounded hidden coordinate space where the internal representation dimension is $D_{\text{host}} = 3072$.

To accurately represent the computational load and physical memory boundaries of FusionLM-v0.1-alpha, we decouple the total parameters residing in memory from the active parameters executed during a forward pass. Because the grafted Qwen and Gemma deltas are injected via localized matrix addition ($W_{\text{host}} + \alpha\tilde{\tau}$), they do not expand the host’s absolute parameter count. The architectural footprint is defined as follows:

- **Total Stored Parameters (P_{total}):** 14.82B (11.0B Vision + 3.8B Text + 23.5M Bridge). This represents the absolute VRAM allocation floor during instantiation.
- **Active Text Parameters ($P_{\text{active_text}}$):** 3.8B. During autoregressive text generation, the vision encoder is entirely bypassed.
- **Active Vision Parameters ($P_{\text{active_vision}}$):** 14.82B. During the initial visual prefill phase, the image is routed sequentially through the full Llama encoder, the projection bridge, and the Phi-4 textual backbone.

By explicitly defining $P_{\text{active_text}}$, we clarify that our benchmarked autoregressive generation speed (24.0 TPS) reflects the execution of a 3.8B dense text graph, while the 14.37 GB VRAM peak reflects the P_{total} static memory constraint.

3.2 Asymmetric Cross-Modal Translation (Llama-3.2 Vision Integration)

Let the frozen source visual network, Llama-3.2-11B-Vision-Instruct, be denoted as E_v . For a given visual input, the encoder processes the image patches into a continuous sequence of high-dimensional embeddings. The output hidden state tensor is defined as:

$$h_v \in \mathbb{R}^{B \times S_v \times D_v}$$

where $D_v = 7680$.

To couple this massive visual space with the $D_{\text{host}} = 3072$ textual space of Phi-4-mini without modifying the pre-trained internal weights of either model, we introduce an isolated, non-bijective projection tensor:

$$W_{\text{bridge}} \in \mathbb{R}^{7680 \times 3072}$$

To improve numerical stability and reduce the likelihood of early amplification or collapse of the projected visual activations prior to any multi-modal continuous pre-training, W_{bridge} is initialized using Orthogonal Initialization.

The forward pass mapping the visual manifold directly into the textual residual stream is parameterized as:

$$h_{\text{textualized}} = h_v W_{\text{bridge}}$$

This linear transformation maps the 7680-dimensional vision output tensor exactly onto the 3072-dimensional boundary of the text model.

3.3 Task Vector Extraction (Qwen and Gemma Donors)

The core structural novelty of our architecture lies in injecting parameter deltas derived from the specializations of Qwen2.5-Coder and Gemma-2-Math. However, natively transplanting parameter matrices from these models into the Phi-4-mini backbone is impossible due to strict structural mismatch ($D_{\text{qwen}} \neq D_{\text{host}}$ and $D_{\text{gemma}} \neq D_{\text{host}}$).

To extract the pure parameter shifts associated with these domains without the structural baggage of the donor models, we utilize Task Vector isolation. This operation is mathematically undefined unless the tuned donor and the base model share perfect architectural and initialization isomorphism. Therefore, we rigidly enforce checkpoint compatibility.

For the coding component, we extract the structural task delta (τ_{code}) utilizing Qwen2.5-Coder and Qwen2.5-Base:

$$\tau_{\text{code}} = W_{\text{tuned}}^{\text{Qwen-Coder}} - W_{\text{base}}^{\text{Qwen}}$$

Similarly, for the mathematical component, we enforce checkpoint isomorphism to extract the delta (τ_{math}):

$$\tau_{\text{math}} = W_{\text{tuned}}^{\text{Gemma-Math}} - W_{\text{base}}^{\text{Gemma}}$$

3.4 Rank-Truncated SVD and Explicit Dimensional Mapping

Once the raw task vectors (τ_{code} and τ_{math}) are extracted, they remain dimensionally incompatible with the host architecture. To resolve this, we employ a dimensional alignment protocol utilizing Singular Value Decomposition (SVD) coupled with an explicit projection operator.

Taking a generalized donor task vector τ_{donor} , we decompose the matrix:

$$\tau_{\text{donor}} = U \Sigma V^T$$

We define a truncation operator that isolates the top k singular values:

$$\tilde{\tau}_{\text{truncated}} = U_k \Sigma_k V_k^T$$

To quantify the structural geometry retained after this dimensional reduction, we actively measure the Explained Variance Ratio (ρ) of the retained singular values:

$$\rho = \frac{\sum_{i=1}^k \sigma_i}{\sum_{i=1}^n \sigma_i}$$

To physically graft this matrix into the host graph, we introduce an explicit affine projection mapping operator, $\mathcal{P}$, which linearly projects and reshapes the truncated delta into the strict boundaries required by the target Phi layer:

$$\tilde{\tau}_{\text{aligned}} = \mathcal{P}(\tilde{\tau}_{\text{truncated}})$$

Specifically, we define $\mathcal{P}(X) = AXB$, where $A \in \mathbb{R}^{d_{\text{host_in}} \times d_{\text{donor_in}}}$ and $B \in \mathbb{R}^{d_{\text{donor_out}} \times d_{\text{host_out}}}$ are static projection matrices initialized to map the donor’s coordinate space directly into the host’s dimensional shape. This mapping resolves the dimension-mapping gap between donor and host execution graphs.

3.5 Deep Layer Structural Integration ($L_{28} - L_{31}$)

We specifically target late-stage intermediate blocks (L_{28} through L_{31}). We hypothesize that late-stage layers may better tolerate dimensional compression due to their operation on higher-level representations.

Let $W_{\text{host}}^{L_i}$ represent a native parameter matrix within a target deep layer. We physically integrate the specialized delta via a localized scaling hyperparameter α (empirically bound at 0.45):

$$W_{\text{grafted}}^{L_i} = W_{\text{host}}^{L_i} + \alpha(\tilde{\tau}_{\text{aligned}})$$

Crucially, because $\tilde{\tau}_{\text{aligned}}$ was mathematically mapped into the host’s dimensional boundaries, the grafted layers maintain exactly 100,669,440 parameters per block, preserving the structural integrity required to bypass CUDA allocation faults during PyTorch compilation.

3.6 Dynamic Graph Stabilization and Idempotent Patches

Compiling this heterogeneous composite architecture requires circumventing strict validation checks. We deploy dynamic monkey-patching at runtime to stabilize the execution graph:

1. **Meta-Tensor Re-Initialization:** The grafting process occasionally orphans positional embedding tensors. If a tensor corresponds to Rotary Position Embeddings (RoPE), we recalculate the frequencies natively:

$$\theta_i = \theta_{\text{base}}^{-\frac{2i}{D_{\text{head}}}}$$

2. **Tied Weights Map Alignment:** We intercept the `post_init()` phase and convert legacy list arrays into symmetric dictionary mappings:

$$\{k : k \mid k \in W_{\text{tied_list}} \wedge \text{type}(k) == \text{str}\}$$

3. **Keyword Argument Boxing:** We override the strict validation of `transformers.utils.LossKwargs` with a generic `TypedDict` shell to prevent inheritance collisions.

4 Feasibility Validation and Empirical Results

We evaluate our grafting methodology against three strict criteria: structural tensor mapping verification, empirical hardware execution viability, and structural graft participation versus a low-rank random control.

4.1 Structural Tensor Mapping and Hardware Benchmarking

The structural audit confirmed that the W_{bridge} tensor was instantiated with a strict non-bijective shape of [7680, 3072], successfully intercepting the visual pipeline. Furthermore, the diagnostic tool isolated the deep intermediate transformer blocks targeted for task injection, verifying a localized footprint of exactly 100,669,440 parameters per layer.

To test operational stability, we prompted the model into sustained, 150-token autoregressive generation loops across varied contexts. Across all inference generations, the model maintained a strict, predictable peak VRAM allocation of 14.37 GB. The active $P_{\text{active_text}}$ architecture sustained a stable average throughput of ~ 24.2 tokens per second (TPS), verifying that the projection bridge and grafted layers did not produce observable throughput degradation.

Table 1: Inference Performance on 15GB VRAM (Nvidia T4)

Test Iteration	Target Output	Latency (s)	Throughput (TPS)	Peak VRAM (GB)
Run 1	150 tokens	6.11	24.54	14.37
Run 2	150 tokens	6.19	24.22	14.37
Run 3	150 tokens	6.21	24.14	14.37
Run 4	150 tokens	6.24	24.03	14.37

4.2 Graft Participation Verification via Activation Divergence

To verify that the dimensionally mapped grafted layers actively participate in the forward computation pass—and that this participation preserves more structured geometric information than an arbitrary algebraic perturbation—we execute a controlled Activation Divergence metric (D_i).

Any low-rank modification to weights natively causes activation divergence. To isolate whether the specific donor geometry of CALG drives the divergence, we execute parallel inference passes using the un-grafted Phi-4-mini host, the FusionLM hybrid, and a *Random Low-Rank Control* ($\mathcal{R}$). The control replaces the task-vector delta with a randomized matrix sharing the exact rank (k) and dimensional boundaries of the CALG projection:

$$\mathcal{R} = U_{\text{rand}} \Sigma_k V_{\text{rand}}^T$$

We measure the L_2 norm of the hidden states at each layer boundary i :

$$D_i^{\text{CALG}} = \|h_i^{\text{Fusion}} - h_i^{\text{Phi}}\|_2$$

$$D_i^{\text{Control}} = \|h_i^{\text{Random}} - h_i^{\text{Phi}}\|_2$$

Empirical observation confirms that while activation divergence remains negligible across the early, un-grafted layers ($L_0 - L_{27}$), D_i^{CALG} spikes significantly at the entry point of the Gemma-Math injection (L_{28}). Crucially, the divergence profile of D_i^{CALG} exhibits a distinct geometric signature compared to the perturbation observed in D_i^{Control} . This explicit deviation establishes that the grafted, projected deltas actively participate in the residual stream with non-random, donor-derived geometric structure.

5 Discussion: Representation Space Mismatch

While the empirical hardware benchmarks confirm that the FusionLM-v0.1-alpha architecture is physically stable, executionally idempotent, and actively computing structured vectors, an analysis of the autoregressive output confirms that the model fails to perform successful zero-shot capability transfer. During sustained generation tasks, the model produces severe character repetitions and semantic fragmentation.

We attribute this failure to fundamental latent manifold mismatch occurring deep within the residual stream, long before logits are mapped to the vocabulary space.

5.1 Latent Manifold Mismatch and Residual Stream Corruption

Neural networks learn representations on distinct, high-dimensional topological manifolds. The Llama-3.2 vision encoder, the Qwen2.5 parameter blocks, the Gemma-2 parameter blocks, and the Phi-4-mini text core were optimized on completely isolated data distributions using disparate initialization seeds. Consequently, their internal geometric vector spaces are fundamentally off-manifold relative to one another.

While our SVD-based dimensional alignment and explicit affine projection ($\mathcal{P}$) successfully force the mathematical boundaries of these tensors to match the host graph’s matrix multiplication requirements, it does not align their *semantic* representations. When the continuous residual stream passes from a native Phi-4 layer into the grafted Qwen layer (L_{30}), the Qwen attention heads interpret the Phi-4 hidden states as out-of-distribution geometric noise. The grafted block performs its computation and projects structurally sound but representationally incompatible vectors back into the residual stream, corrupting the attention key/query projections and irreparably fracturing the context before it reaches the final language modeling

head.

5.2 The Pre-Aligned Structural Primitive

Because the hidden states are already off-manifold, any subsequent vocabulary-space mapping is fundamentally incoherent. Therefore, FusionLM-v0.1-alpha must be classified strictly as a **Pre-Aligned Structural Primitive**. CALG demonstrates that components originating from structurally incompatible foundation models can be projected into a common tensor geometry, inserted into a shared execution graph, and executed stably on commodity hardware, even though semantic alignment is not achieved without subsequent continuous adaptation.

6 Conclusion and Future Work: Boundary-Only Healing

We have introduced Cross-Architecture Layer Grafting (CALG), a systems-architecture methodology for synthesizing heterogeneous neural graphs. We have proven that massive vision modules and task-vector deltas can be dynamically grafted into a compact text backbone and co-compiled inside highly restrictive consumer VRAM profiles.

As hypothesized in our feasibility testing, the raw structural graft natively induces severe representation-space mismatch. The physical scaffolding has been successfully built, but the architectural boundaries remain semantically isolated.

A critical next step is a targeted boundary-only latent healing experiment. Future work will lock the primary parameters of the host and vision encoder (θ_{frozen}), applying Parameter-Efficient Fine-Tuning (PEFT) strictly to the structural boundaries: the cross-modal bridge (W_{bridge}) and the dimensionally grafted task blocks ($L_{28} - L_{31}$).

By training these localized boundaries for a brief duration (e.g., 10,000–50,000 steps) with Low-Rank Adaptation (LoRA), we can evaluate whether localized adaptation partially restores semantic coherence and recovers interoperability at a fraction of standard multi-modal alignment costs.

References

- [1] Alayrac, J.-B., et al. (2022). Flamingo: a visual language model for few-shot learning. *Advances in Neural Information Processing Systems*, 35, 23716–23736.
- [2] Ilharco, G., et al. (2022). Editing models with task vectors. *ICLR*. arXiv preprint arXiv:2212.04089.
- [3] Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual instruction tuning. *Advances in Neural Information Processing Systems*, 36.
- [4] Men, X., et al. (2024). ShortGPT: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*.
- [5] Yadav, P., et al. (2023). TIES-Merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36.
- [6] Yu, L., et al. (2024). Language models are super mario: Absorbing abilities from homologous models as a free lunch. *ICML*. arXiv preprint arXiv:2311.03099.